

Informe TP N°6

- **Objetivo del TP:** Obtener los conocimientos necesarios para poder utilizar un analizador de trafico
- **Materiales:** WireShark
- **Escenario:** Analizaremos logueos HTTP y segmentos TCP/UDP con un analizador de trafico como lo es WireShark
- **Actividad:**
 1. ¿Para qué sirve WireShark?
 2. Desarrollar un ejemplo de cómo la contraseña de un login HTTP puede ser vulnerado con la herramienta.
 3. Analizar un segmento TCP de forma aleatoria e identificar todos los elementos del formato del segmento. Los mismos tienen que estar expresados en formato decimal.
 4. Analizar un segmento UDP de forma aleatoria e identificar todos los elementos del formato del segmento. Los mismos tienen que estar expresados en formato decimal.
 5. Desarrollar una conclusión del trabajo realizado.

1) ¿Para qué sirve WireShark?

Wireshark sirve para analizar el tráfico de una red. Con esta herramienta se pueden capturar los paquetes que circulan por la red y verlos en detalle. Es útil para detectar errores, ver cómo funcionan ciertos protocolos o incluso encontrar posibles fallas de seguridad. También se puede usar para aprender cómo se comunican los dispositivos en una red.

En seguridad también se usa para detectar comportamientos extraños o posibles ataques. Por ejemplo, si alguien está enviando muchos paquetes a un puerto específico o si hay contraseñas viajando sin cifrado.

Además podés filtrar por direcciones IP, protocolos, puertos, etc., para encontrar paquetes específicos dentro de todo el tráfico capturado.

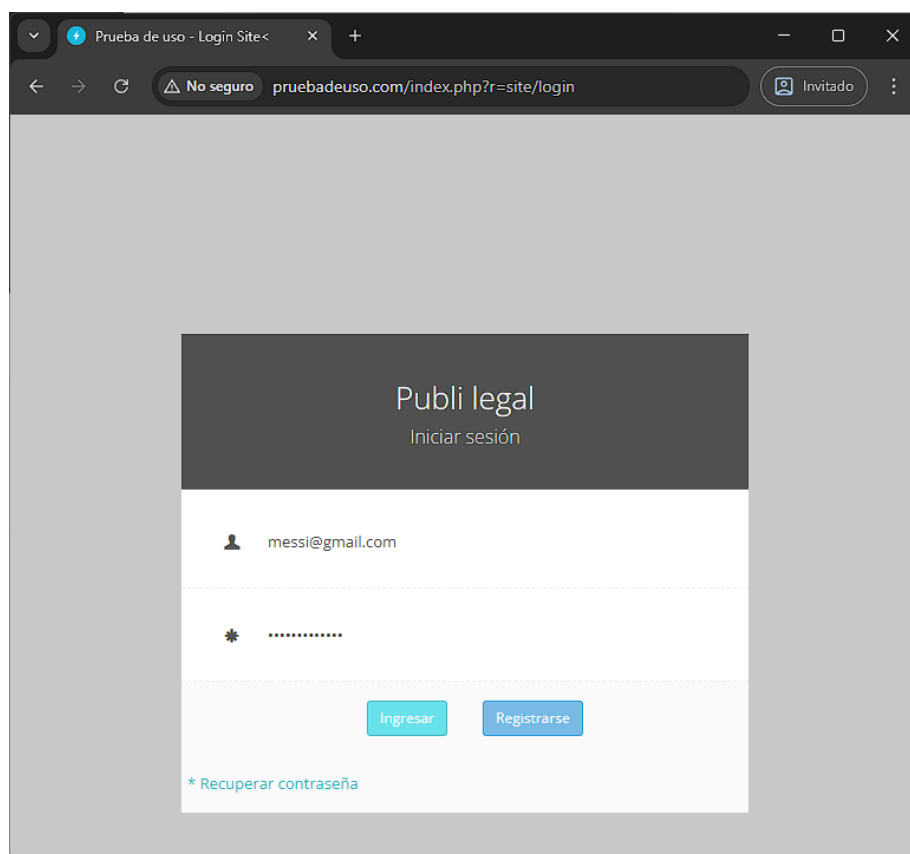
2) Ejemplo de vulnerabilidad de Contraseña con HTTP

Se puede ver las claves de los usuarios y sus mails con suma facilidad, ya que el protocolo HTTP usa el puerto 80 que no va cifrado. En cambio el HTTPS si usa cifrado para los paquetes e impidiendo que no se pueda ver el tráfico.

Ejemplo:

Realizamos un acceso a un sitio web con el protocolo HTTP, ingresamos las credenciales y observamos los paquetes y los resultados que capturó Wireshark.

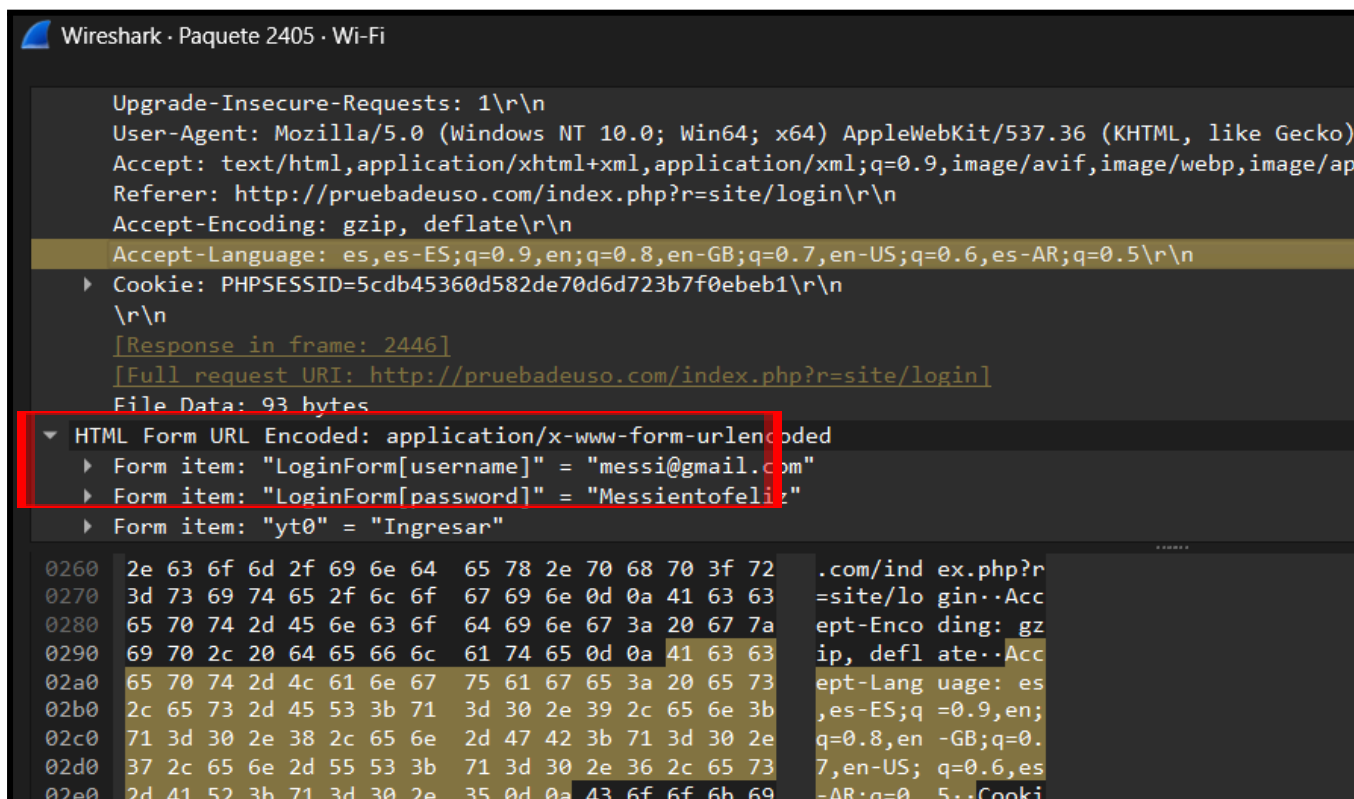
Como primer paso es necesario capturar el tráfico de nuestra interfaz de red, en nuestro caso Ethernet, luego en nuestro navegador ingresamos a un sitio web que utilice el protocolo HTTP, nosotros elegimos el sitio pruebadeuso.com para este ejemplo. Una vez que accedimos, ingresamos datos fake en los campos del formulario para realizar la prueba.



Después en el filtro que nos brinda la herramienta, aplicamos un filtro por el protocolo HTTP y nos enfocamos en el paquete con el método HTTP POST que es el que corresponde al envío del formulario.

```
HTTP 807 POST /index.php?r=site/login HTTP/1.1 (application/x-www-form-urlencoded)
```

Una vez ubicado el paquete, buscamos el elemento con el nombre “HTML Form URL Encoded: application/x-www-form-urlencoded” y dentro podemos observar lo siguiente:



The image shows a Wireshark packet capture of an HTTP POST request. The packet is selected, and the 'Form Data' section is expanded, showing the following items:

- Form item: "LoginForm[username]" = "messi@gmail.com"
- Form item: "LoginForm[password]" = "Messientofeliz"
- Form item: "yt0" = "Ingresar"

The packet details pane also shows the following headers:

```

Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/ap
Referer: http://pruebadeuso.com/index.php?r=site/login\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: es,es-ES;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6,es-AR;q=0.5\r\n
Cookie: PHPSESSID=5cdb45360d582de70d6d723b7f0eb1\r\n
[Response in frame: 2446]
[Full request URI: http://pruebadeuso.com/index.php?r=site/login]
File Data: 93 bytes
  
```

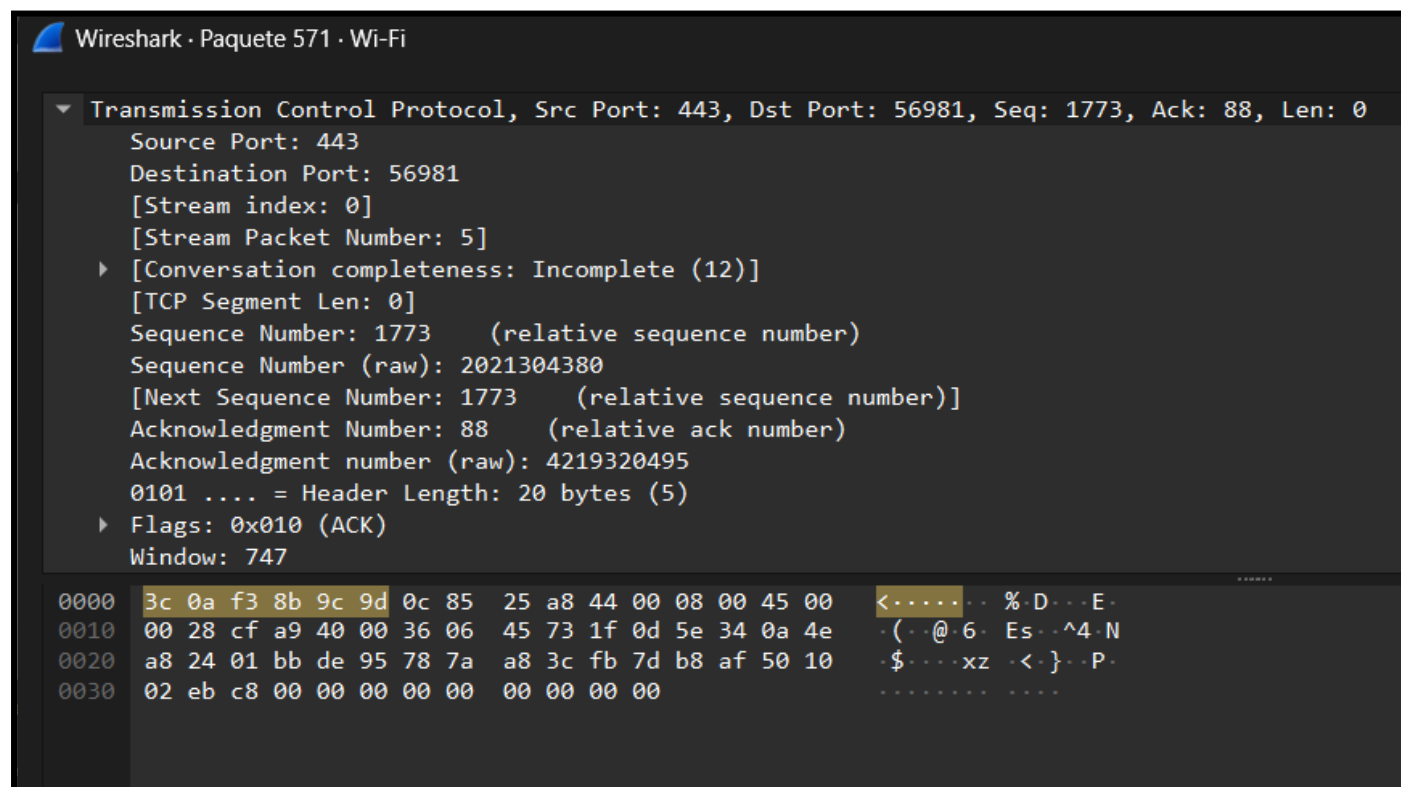
Como podemos observar, nos muestra las credenciales tal como las ingresamos:

User: messi@gmail.com

Password: Messientofeliz

De esta forma se puede vulnerar las credenciales de un usuario sin ningún tipo de esfuerzo, con solamente capturar el paquete que le corresponde al envío del formulario de sitio HTTP, cualquier persona con malas intenciones tendría acceso a nuestra información sensible sin ningún tipo de problema y esfuerzo.

3) Analizamos los elementos del formato del segmento TCP, como se ve está en decimal



En el segmento TCP se encuentran los siguientes campos

Encabezado TCP (Transmission Control Protocol):

Source Port (Puerto Origen): 443

Es el puerto desde el cual se origina la conexión (443 = HTTPS).

Destination Port (Puerto Destino): 56981

Es el puerto en el host destino que recibe la conexión (puerto efímero del cliente).

[Stream index: 0]

Identificador de la conversación TCP dentro de Wireshark.

[Stream Packet Number: 5]

Número del paquete dentro de la secuencia de ese flujo TCP.

Conversation completeness: Incomplete (12)

Indica que Wireshark detectó que la conversación TCP está incompleta.

Control de secuencia:

Sequence Number (Número de Secuencia): 1773 (relativo)
Número usado para ordenar los segmentos.

Sequence Number (raw): 2021304380
Valor absoluto real del número de secuencia.

Next Sequence Number: 1773 (relativo)
Próximo número esperado (como no hay datos, queda igual).

Acknowledgment Number (Número de Acuse): 88 (relativo)
Indica el siguiente byte que espera recibir el emisor.

Acknowledgment Number (raw): 4219320495
Valor absoluto real.

Cabecera TCP:

Header Length: 20 bytes (5 palabras de 32 bits)
El encabezado ocupa 20 bytes, lo mínimo posible.

Flags: 0x010 (ACK)
Bandera activa: ACK (se está reconociendo recepción de datos).

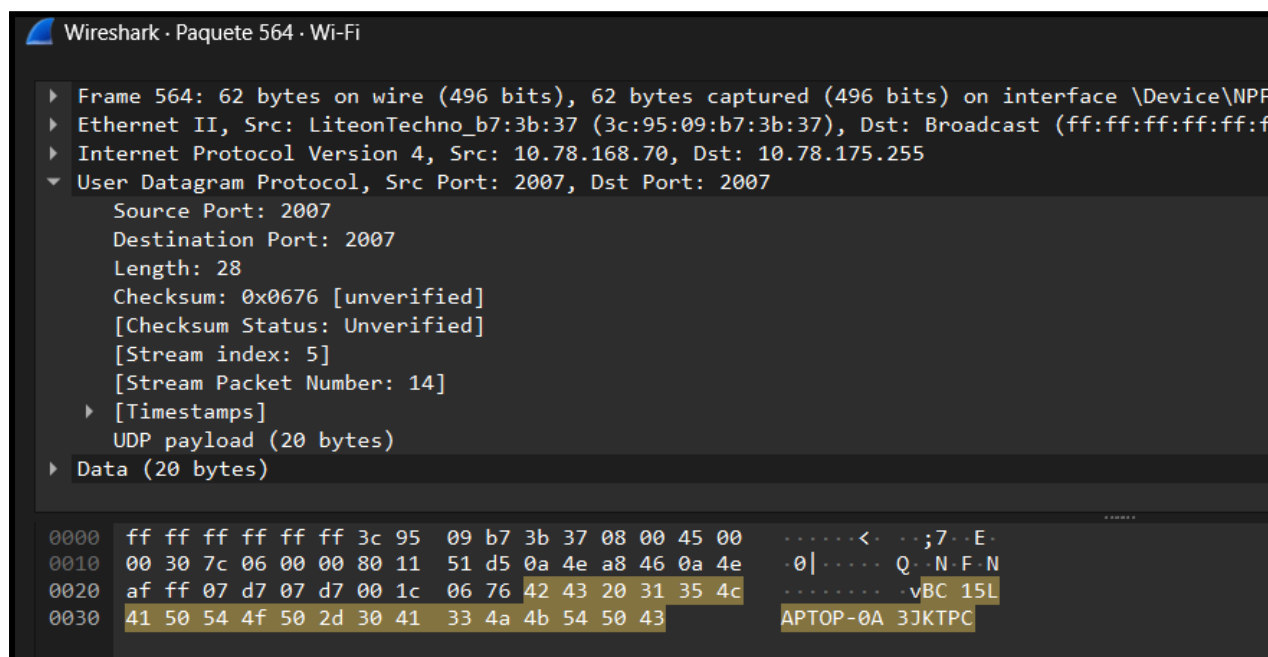
Window Size Value: 747
Tamaño de la ventana de recepción (cuántos bytes puede recibir antes de necesitar ACK).

Datos crudos (Raw Data - Hex dump en la parte inferior)

3c 0a f3 8b 9c 9d ...

Es la representación hexadecimal de los bytes del paquete.
A la derecha aparece la interpretación ASCII (cuando es imprimible).
En este caso, son principalmente datos cifrados ya que el puerto es 443 (HTTPS).

4) Ahora analizamos un segmento aleatorio de UDP



```

Wireshark · Paquete 564 · Wi-Fi

▶ Frame 564: 62 bytes on wire (496 bits), 62 bytes captured (496 bits) on interface \Device\NPF...
▶ Ethernet II, Src: LiteonTechno_b7:3b:37 (3c:95:09:b7:3b:37), Dst: Broadcast (ff:ff:ff:ff:ff:f
▶ Internet Protocol Version 4, Src: 10.78.168.70, Dst: 10.78.175.255
▼ User Datagram Protocol, Src Port: 2007, Dst Port: 2007
    Source Port: 2007
    Destination Port: 2007
    Length: 28
    Checksum: 0x0676 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 5]
    [Stream Packet Number: 14]
    ▶ [Timestamps]
    UDP payload (20 bytes)
▶ Data (20 bytes)

0000  ff ff ff ff ff ff 3c 95 09 b7 3b 37 08 00 45 00  .....<...;7..E.
0010  00 30 7c 06 00 00 80 11 51 d5 0a 4e a8 46 0a 4e  .0|.....Q..N.F.N
0020  af ff 07 d7 07 d7 00 1c 06 76 42 43 20 31 35 4c  .....vBC 15L
0030  41 50 54 4f 50 2d 30 41 33 4a 4b 54 50 43      APTOP-0A 3JKTPC
  
```

Formato de segmento UDP analizado

El encabezado UDP siempre tiene 8 bytes con 4 campos principales.

Puerto origen: 2007

Puerto destino: 2007

Longitud: 28 (incluye header UDP de 8 bytes + 20 bytes de datos)

Checksum: 0x0676 = 1654

Datos (Payload): 20 bytes

Tabla de valores en decimal

Campo	Valor (decimal)
Puerto origen	2007
Puerto destino	2007
Longitud	28
Checksum	1654
Datos (Payload)	20 bytes

5) CONCLUSIÓN

Con este trabajo aprendimos bastante sobre cómo funciona una red por dentro, y cómo se puede ver y capturar todo lo que pasa usando el software Wireshark.

Descubrimos que Wireshark sirve para capturar y analizar el tráfico de red, y que se pueden ver cosas muy detalladas, incluso datos sensibles como usuarios y contraseñas si la conexión no está cifrada, como en HTTP.

Cuando analizamos un segmento TCP, observamos que tiene varios datos importantes como los puertos, los números de secuencia, las banderas (flags), etc. Todos esos datos ayudan a que la conexión sea confiable, lleguen correctamente y en orden los mensajes. Todo estaba en base decimal a la hora de verlo.

Luego realizamos los mismos pasos con un segmento UDP y pudimos observar que es mucho más simple, pero no garantiza que el mensaje llegue. Por eso se usa más en cosas en vivo como juegos o videollamadas, donde lo importante es que llegue la información lo antes posible, priorizando velocidad aunque se pueda perder información.

Gracias a las herramientas y los puntos realizados del trabajo logramos entender cómo se ven los datos de las redes y toda la información que circula en un dispositivo, viendo todo tipo de protocolos y paquetes. Incluso se dio el pie para investigar temas de seguridad como https y temas relacionados como la distribución de Linux Kali.